

Evidence-Guided Kubernetes Upgrade Readiness: A Bounded-Write Operator Artifact and Controlled Benchmark

Ihsen Alaya

Abstract—Kubernetes upgrades are recurring software evolution activities in which platform changes interact with application APIs, disruption budgets, admission controls, workload availability assumptions, scheduling capacity, operators, add-ons, and observability capabilities. Existing operational tools are useful for detecting deprecated Kubernetes APIs, but upgrade readiness also depends on whether workloads can tolerate node disruption, whether admission policies will allow recreated objects, whether platform engineers can collect sufficient evidence, and whether remediation actions are traceable to concrete cluster state. This paper presents KubeUpgrade Guardian, a Kubernetes-native bounded-write operator artifact for evidence-guided upgrade readiness assessment. The artifact implements two custom resources, `UpgradeAssessment` and `UpgradePlan`, and deterministic checkers that produce structured findings, evidence records, aggregate risk scores, non-executing decisions, and prioritized remediation actions. The operator does not upgrade clusters, drain nodes, patch workloads, or apply remediations; it reads assessed resources and writes only its own assessment status and generated plan resources. We report implementation fidelity, local validation from `go test ./...`, and a controlled Kind benchmark with 31 labeled findings and 5 explicit negative controls across deprecated API, workload availability, readiness, PDB, admission, and selected policy-risk families. In run 20260617T091703Z, KubeUpgrade Guardian matched all 31 labeled findings with 0 false positives and 0 false negatives, produced 0 observations on all negative controls, and generated a `UpgradePlan` with 31 non-executing required actions. Pluto and kubent matched all 4 deprecated-API findings in the same benchmark; they are treated as specialized API-deprecation baselines and are not credited or penalized as general readiness tools. On the 27 non-API readiness labels, KubeUpgrade Guardian covered 27/27, while the API-focused baselines covered 0/27 by scope.

INDEX TERMS

Kubernetes, software evolution, upgrade readiness, cloud-native systems, DevOps, operators, impact analysis, evidence-guided maintenance, empirical software engineering, AIOps.

I. INTRODUCTION

Kubernetes has become a common substrate for operating cloud-native software systems, but a production cluster is not merely an execution platform. It is a continuously

evolving socio-technical system composed of workloads, controllers, custom resources, admission policies, resource constraints, identity rules, observability pipelines, and managed-provider services. Upgrading such a system is a software evolution problem: a platform version change can alter the validity, behavior, or operational assumptions of dependent artifacts [1], [3], [2]. The practical challenge is not only whether the target Kubernetes version exists. The challenge is whether the specific cluster can be upgraded without exposing application availability, policy, capacity, compatibility, or observability gaps.

The Kubernetes ecosystem already provides authoritative upgrade primitives and policies. The Kubernetes documentation defines API deprecation, removed APIs, version-skew constraints, API-initiated eviction, `PodDisruptionBudget` behavior, admission webhooks, `ValidatingAdmissionPolicy`, custom resources, RBAC, and the operator pattern [22], [23], [24], [25], [26], [27], [29], [32], [30], [33]. Managed platforms such as AKS add provider-specific upgrade options, node-image mechanics, upgrade channels, and day-2 practices [37], [38], [39]. These documents are necessary, but they do not by themselves turn a live cluster into a structured, evidence-backed readiness assessment.

Operational teams often complement official guidance with tools such as Pluto, kube-no-trouble (kubent), Helm API migration documentation, and Helm metadata repair tools [40], [41], [42], [43]. These tools are valuable, especially for API removals, but they focus on a subset of readiness risk. A cluster can have no removed API versions and still fail operationally because a single-replica workload is disrupted, a `PodDisruptionBudget` blocks node drain, an admission webhook is unavailable, a restricted namespace rejects recreated pods, a node pool lacks one-node-loss headroom, or RBAC prevents the platform team from collecting the evidence needed to make a decision.

This paper argues that Kubernetes upgrade readiness should be treated as evidence-guided impact analysis over live cluster state. Rather than producing only warnings or checklist items, an assessment should generate findings that identify affected resources, observable evidence, severity, operational impact, and a non-executing remediation recommendation. Those findings should be materialized in a Kubernetes-native artifact so that platform engineers can review, audit, store, and compare assessments before upgrade execution.

We instantiate this idea in KUBEUPGRADE GUARDIAN, a read-only-by-design operator artifact. The current implementation exposes `UpgradeAssessment` and `UpgradePlan`; findings and evidence are structured fields inside those resources, not standalone CRDs. The artifact implements deterministic checkers for workload availability, missing readiness probes, PodDisruptionBudget risks, admission webhooks, policy risks, deprecated or removed APIs, capacity drain risk, observability gaps, and RBAC assessment gaps. It computes aggregate scores and decisions, then creates a prioritized `UpgradePlan` without executing any disruptive action.

The paper makes four scoped contributions.

- 1) An evidence-guided model for Kubernetes upgrade readiness assessment that links risk categories to observable evidence and non-executing remediation actions.
- 2) A bounded-write Kubernetes operator artifact implementing `UpgradeAssessment`, `UpgradePlan`, deterministic checkers, scoring, decisions, and idempotent plan generation.
- 3) An implementation-fidelity analysis that maps manuscript claims to implemented, locally tested, and benchmark-observed artifact behavior.
- 4) A reproducible controlled Kind benchmark comparing KUBEUPGRADE GUARDIAN with Pluto and kubent on labeled upgrade-readiness scenarios, explicit negative controls, archived raw outputs, and measured detection tables.

II. BACKGROUND AND PROBLEM STATEMENT

A. Upgrade Readiness as Software Evolution

Software evolution research frames long-lived software as a system that must change while preserving useful behavior [1], [3]. Change impact analysis studies how to identify the artifacts affected by a proposed change before that change is made [2]. Kubernetes upgrades fit this pattern: the changed artifact is the platform version, but the affected artifacts include API objects, workloads, admission controllers, add-ons, RBAC permissions, resource requests, and operational workflows.

DevOps and continuous delivery literature emphasizes fast, reliable, automated change delivery, but also documents adoption barriers and the need for reliable feedback loops [4], [5], [6], [7], [8], [9]. SRE practice similarly emphasizes measurable reliability, production readiness, and operational learning [10], [11]. In a Kubernetes upgrade, readiness assessment is one such feedback loop: it should convert cluster state into evidence that engineers can act on before scheduling disruptive work.

B. Kubernetes-Specific Upgrade Mechanics

Kubernetes upgrades are constrained by API deprecation and removal schedules, component version skew, eviction behavior, PodDisruptionBudgets, admission control,

RBAC, and resource scheduling. The Kubernetes deprecation guide documents removed APIs by release, while the deprecation policy explains the compatibility process [22], [23]. Version-skew policy constrains how far cluster components may differ during upgrades [24]. API-initiated eviction, including the behavior used by `kubect1 drain`, interacts with PodDisruptionBudgets and termination behavior [25], [26]. Admission webhooks and ValidatingAdmissionPolicy can reject newly created or updated objects during post-upgrade reconciliation [27], [29]. Pod Security Admission and Pod Security Standards add another policy layer that may reject workload templates after platform or policy changes [34].

C. Problem Statement

The problem addressed by this paper is:

Given a Kubernetes cluster and a target Kubernetes minor version, how can platform engineers obtain a repeatable, auditable, evidence-backed readiness assessment that covers more than API deprecations while avoiding unsafe automated remediation?

This paper evaluates the bounded artifact at two levels. First, implementation fidelity is checked by mapping claims to the actual CRDs, controller behavior, deterministic checkers, score computation, and generated plan resources. Second, controlled detection behavior is measured against version-pinned Kind scenarios with labeled positive findings and explicit negative controls. The study does not claim production-cluster generalization; it reports reproducible scenario-level evidence for the implemented artifact.

III. KUBERNETES UPGRADE READINESS RISK FAMILIES

Upgrade readiness is not a single property. It is a set of risk families that interact during upgrade preparation and execution. Table I summarizes the risk taxonomy used by KUBEUPGRADE GUARDIAN.

The taxonomy is intentionally broader than deprecated APIs. API-removal tools are necessary baselines, but an upgrade may be blocked by disruption semantics, capacity, admission behavior, or insufficient permissions. Treating RBAC denial as an assessment finding is particularly important: an assessment that cannot inspect critical resource types should not silently produce a green result.

The controlled benchmark prioritizes depth in the families that can be represented as deterministic Kind fixtures: deprecated APIs, workload availability/readiness, PDB and eviction behavior, and admission/policy indicators. Capacity, observability, add-on compatibility, and managed-provider constraints remain part of the artifact taxonomy, but they are not used as measured accuracy claims in this benchmark.

IV. EVIDENCE-GUIDED ASSESSMENT MODEL

A. Core Data Model

The model is organized around four concepts.

TABLE I
KUBERNETES UPGRADE READINESS RISK FAMILIES.

Risk family	Example failure mode	Observable evidence	Current artifact coverage
API evolution	Object uses an API version removed by the target minor release	Group/version/kind, removed-in release, target version	Implemented static MVP table
Workload availability	Critical workload has fewer than two replicas or runs as a standalone pod	Replica count, owner references, namespace scope	Implemented
Readiness	Recreated pod receives traffic before it is ready or cannot signal readiness	Container readiness probe presence	Implemented
PDB/eviction	PDB prevents voluntary disruption during drain	PDB selector, replica count, minAvailable/maxUnavailable	Implemented
Admission and policy	Webhook, ValidatingAdmissionPolicy, or Pod Security setting rejects recreated objects	Webhook configuration, missing services, restricted labels, pod security fields	Partially implemented
Capacity	Cluster cannot absorb one-node-loss or surge capacity during node upgrade	Node allocatable resources, pod requests, remaining headroom	Implemented conservative check
Observability	Upgrade blockers cannot be observed or validated	Monitoring namespaces, Prometheus CRDs, telemetry hints	Implemented heuristic
RBAC and evidence gaps	Assessment cannot inspect required resources	Forbidden API operations and denied resource types	Implemented as findings
Add-on/operator compatibility	Controller or CRD compatibility with target release is unknown	CRDs, Helm releases, controller images, support matrix	Out of current benchmark
Managed-provider constraints	AKS node-pool, image, channel, or surge settings shape feasible plans	Provider metadata, node pool settings, AKS upgrade options	Out of current benchmark

Evidence.

An observable fact collected from the Kubernetes API, such as a replica count, PDB selector, admission webhook failure policy, removed API version, resource request total, or RBAC denial.

Finding.

A risk or assessment gap backed by one or more evidence records. A finding includes an identifier, type, severity, category, optional Kubernetes resource reference, message, evidence, and recommendation.

Assessment.

A bounded snapshot of readiness for a target Kubernetes version and namespace scope. In the artifact, this is materialized as `UpgradeAssessment`.

Plan.

A non-executing remediation artifact that orders required actions and carries the aggregate risk decision. In the artifact, this is materialized as `UpgradePlan`.

B. Scoring and Decisions

The current artifact uses an intentionally simple scoring model. Critical findings add 25 points, high findings add 10, medium findings add 4, low findings add 1, and informational findings add 0. Aggregate risk is low below 11, medium from 11 to 30, high from 31 to 60, and critical above 60. A single critical finding yields a `DoNotUpgrade` decision. A score above 30 yields `ProceedWithCaution`. A high or critical RBAC assessment gap can yield `AssessmentIncomplete` when higher-priority decision rules do not already apply. Otherwise, the decision is `Proceed`.

This score is not presented as a validated risk model. It is a deterministic prioritization mechanism used to order findings and generate a non-executing plan. The controlled benchmark measures detection accuracy and

negative-control behavior; it does not calibrate severity weights against production outcomes.

C. Non-Execution Principle

KUBEUPGRADE GUARDIAN is designed to support readiness assessment, not upgrade execution. The model forbids the assessment tool from upgrading control planes, draining nodes, patching workloads, modifying policies, changing PDBs, or applying remediations automatically. This follows a conservative SRE posture: the assessment should reduce uncertainty before a risky operation, not become another source of unreviewed change [10], [11].

V. KUBEUPGRADE GUARDIAN OPERATOR

A. Custom Resources

The current artifact implements two CRDs.

- **UpgradeAssessment**: desired assessment target version, mode, namespace scope, selected checks, and observed status including phase, risk level, score, summary, findings, generated plan reference, and conditions.
- **UpgradePlan**: source assessment reference, decision, risk level, score, summary, required non-mutating actions, recommended order, generation timestamp, and conditions.

Findings and evidence are embedded structured fields. They are not standalone CRDs. This is an important fidelity point because a paper that claims separate `UpgradeFinding` or `UpgradeEvidence` resources would not match the implementation.

B. Reconciliation Flow

The controller watches `UpgradeAssessment` resources and owns generated `UpgradePlan` resources. Reconciliation marks the assessment running, executes selected

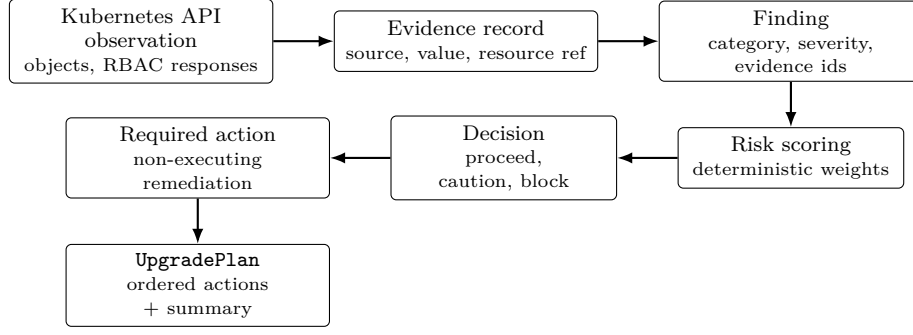


Fig. 1. Evidence pipeline used by the assessment model. Each finding is traceable to collected evidence, and each generated action remains non-executing.

checkers, sorts findings deterministically, computes score and decision, upserts an idempotent plan, and then writes completed assessment status. The controller is a Kubernetes operator in the sense of the Kubernetes operator pattern: it links custom resources to controller behavior without modifying Kubernetes itself [33], [32], [48].

C. Implemented Checkers

Table II maps implemented checkers to evidence and findings.

Admission and policy coverage is deliberately scoped. The evaluated benchmark covers fail-closed webhooks, absent webhook services, broad webhook namespace scope, Pod Security **restricted** enforcement labels, privileged containers, missing security contexts, hostPath volumes, and **allowPrivilegeEscalation=true**. It does not model full admission dry-run semantics, ValidatingAdmissionPolicy expression evaluation, complete Kyverno/Gatekeeper policy semantics, image-policy webhooks, external authorization systems, or provider-managed admission chains.

D. Plan Generation

The planner converts non-informational findings into required actions. Each action includes severity, category, resource reference, remediation text, and evidence references. The default recommended order is: fix critical blockers, fix high-risk workloads, validate admission policies, validate observability, and proceed with controlled upgrade. The order is deterministic and non-executing.

VI. IMPLEMENTATION FIDELITY AND ARTIFACT SCOPE

This section maps experimental claims to implemented artifact behavior.

A. Artifact Fidelity

The manuscript was aligned with the operator artifact used by benchmark run 20260617T091703Z and committed as b11c531. The artifact includes:

- Go API types for **UpgradeAssessment**, **UpgradePlan**, **Finding**, **Evidence**, **RequiredAction**, summary, risk levels, and decisions.

- Generated CRD YAML for **UpgradeAssessment** and **UpgradePlan**.
- A controller-runtime reconciler that reads assessed resources, writes assessment status, and creates or updates a generated plan.
- Eight deterministic checker categories and RBAC gap modeling.
- Scoring, decision, and planner packages.
- Helm packaging and GitOps wiring in the repository.

Table III records which claims are supported by the current artifact, local tests, and controlled benchmark outputs.

Features absent from this matrix, including provider-specific collectors, broad field validation, resource-cost evaluation, and explanation-agent integration, are not used as empirical claims in this manuscript.

B. Scope Boundaries

The artifact scope is intentionally bounded:

- controlled benchmark metrics are produced by the external reproducibility package, not by the controller implementation itself;
- the removed-API checker uses an MVP static table rather than a complete versioned database;
- managed-provider evidence collection for AKS node-pool upgrade settings is outside the measured benchmark dataset;
- add-on support-matrix reasoning for cert-manager, ingress controllers, policy engines, storage drivers, and observability stacks is outside the core checker set;
- explanation-agent behavior is outside the evaluated artifact;
- automatic upgrade, drain, patch, or remediation execution is outside the design.

C. Bounded-Write Semantics

The phrase “read-only” is precise only with respect to assessed cluster resources. The operator does write its own CR status and **UpgradePlan** resources. A stricter description is bounded-write assessment: no assessed workload, node, PDB, policy, webhook, CRD, add-on, or managed-provider resource is mutated by the assessment.

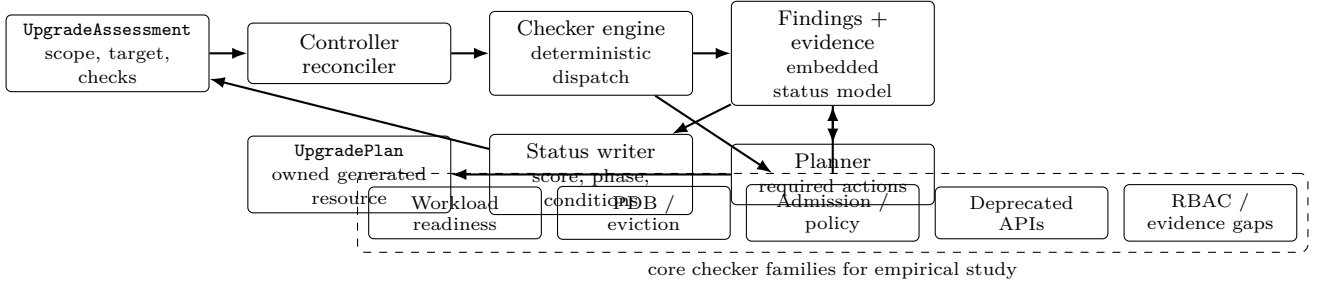


Fig. 2. KUBEUPGRADE GUARDIAN artifact architecture. The reconciler reads assessment scope, dispatches deterministic checkers, materializes evidence-backed findings, and writes bounded assessment outputs.

TABLE II
IMPLEMENTED DETERMINISTIC CHECKERS IN THE CURRENT ARTIFACT.

Checker	Triggered finding type	Evidence examples	Artifact status
Workload availability	WORKLOAD_AVAILABILITY_RISK	Deployment/StatefulSet replicas below two; standalone pod owner references	Implemented
Readiness probes	MISSING_READINESS_PROBE	Container name and missing readiness probe	Implemented
PDB	PDB_BLOCKING_RISK	Replica count, selector, <code>minAvailable</code> , <code>maxUnavailable</code>	Implemented
Admission webhooks	ADMISSION_WEBHOOK_RISK	<code>failurePolicy=Fail</code> , missing webhook service, broad selector	Implemented
Policy risks	POLICY_RISK; POLICY_ENGINE_DETECTED	Pod Security restricted labels, privileged containers, hostPath volumes, Kyverno/Gatekeeper CRDs	Partially implemented
Deprecated APIs	DEPRECATED_OR_REMOVED_API	Static removed API table, target minor version, live object GVK	MVP table only
Capacity	CAPACITY_DRAIN_RISK	Node allocatable CPU/memory, pod requests, one-node-loss headroom	Conservative heuristic
Observability	OBSERVABILITY_GAP; OBSERVABILITY_CAPABILITY_DETECTED	Monitoring namespace and Prometheus CRD presence	Heuristic
RBAC gap helper	RBAC_ASSESSMENT_GAP	Forbidden list/get operations and API resource names	Implemented across checkers

VII. LOCAL ARTIFACT VALIDATION

The local validation objective is artifact sanity, not production effectiveness. The command `go test ./...` was executed against the operator repository and completed with 0 failures. A JSON-formatted run reported 27 passing test events, including subtests, across the checked packages. Packages without tests were reported as such by Go. Table V summarizes the covered behavior.

This test suite supports claims about implemented behavior and deterministic transformations. Controlled recall, false-positive behavior, and baseline coverage are reported separately in Section VIII.

VIII. CONTROLLED BENCHMARK RESULTS

This section reports the executed empirical benchmark. The benchmark is intentionally controlled: it tests whether the current artifact detects labeled upgrade-readiness findings in reproducible Kind scenarios and whether safe fixtures remain silent. It does not estimate production-cluster prevalence.

A. Benchmark Setup

The benchmark package is archived under `experiments/kind/r01-benchmark`. The final validated run is `results/20260617T091703Z`. It used Kind image `kindest/node:v1.24.15` to allow creation of Kubernetes

APIs removed by later releases, target Kubernetes version 1.32, Pluto 5.24.0, and kubent 0.7.3. The Kubernetes client was `v1.34.1` and the server was `v1.24.15`; this skew is recorded in the tool-version artifact and is acceptable for the benchmark because the run exercises server-side legacy API availability rather than client feature behavior.

The ground truth contains 31 expected findings: 4 deprecated-API findings, 3 workload-availability findings, 5 readiness-probe findings, 7 PDB findings, 4 admission-webhook findings, and 8 selected policy-risk findings. It also contains 5 negative controls: a safe three-replica deployment with readiness and PDB coverage, a scoped fail-open webhook with an existing service, a namespace with warn/audit Pod Security labels but no enforce label, a modern `autoscaling/v2` HPA, and a modern `policy/v1` PDB served by a legacy-capable API server. Raw outputs include the `UpgradeAssessment`, generated `UpgradePlan`, baseline reports, normalized findings, negative-control observations, metrics, controller logs, and tool-version records.

B. Risk Family Occurrence

All six targeted risk families were exercised by the controlled scenarios. The generated assessment reported 31 findings: 7 critical, 13 high, 9 medium, and 2 low. The

TABLE III
ARTIFACT FIDELITY MATRIX FOR THE CURRENT MANUSCRIPT CLAIMS.

Feature	Implemented	Locally tested	Used as paper claim	Notes
<code>UpgradeAssessment</code> CRD	Yes	Yes	Core artifact claim	Generated CRD and reconciler input.
<code>UpgradePlan</code> CRD	Yes	Yes	Core artifact claim	Generated plan resource owned by assessment.
Workload availability checker	Yes	Yes	Core checker claim	Detects low-replica and standalone-pod risks.
Readiness probe checker	Yes	Yes	Core checker claim	Detects missing container readiness probes.
PDB checker	Yes	Yes	Core checker claim	Detects blocking disruption-budget patterns.
Admission webhook checker	Yes	Yes	Core checker claim	Detects risky failure policy, missing services, and broad namespace scope.
Policy risk checker	Partial	Yes	Scoped checker claim	Covers selected Pod Security and policy-engine indicators only.
Deprecated API checker	Partial	Yes	Scoped checker claim	Uses a small static MVP removed-API table and last-applied source filtering to avoid API-server conversion false positives.
Capacity checker	Yes	Yes	Core checker claim	Conservative one-node-loss headroom check.
Observability checker	Yes	Yes	Scoped checker claim	Heuristic signal, not proof of monitoring quality.
RBAC gap modeling	Yes	Yes	Core assessment-gap claim	Forbidden API calls become explicit findings.
Scoring and decision logic	Yes	Yes	Deterministic artifact claim	Heuristic thresholds; not empirically calibrated.
Planner	Yes	Yes	Non-executing plan claim	Produces ordered required actions only.

TABLE IV
CLAIMS AND SUPPORTING EVIDENCE IN THE CURRENT MANUSCRIPT.

Claim	Current evidence	Status	Allowed interpretation
The operator creates <code>UpgradePlan</code> resources.	Controller and planner tests; generated CRD; reconciler implementation.	Locally validated	Supported as implemented artifact behavior.
The operator detects low-replica workload scenarios.	Local checker test for low-replica workloads.	Locally validated	Supported for covered fixture behavior.
The operator detects missing readiness probe scenarios.	Local checker test for missing readiness probes.	Locally validated	Supported for covered fixture behavior.
The operator detects a blocking PDB scenario.	Local checker test for a single-replica workload with blocking PDB.	Locally validated	Supported for covered fixture behavior.
The operator detects labeled controlled benchmark findings.	Kind run 20260617T091703Z: 31 TP, 0 FP, 0 FN against benchmark ground truth.	Controlled benchmark	Supported only for the controlled scenarios and labels in the replication package.
The benchmark contains explicit negative controls.	Five negative controls produced 0 observations for KUBEUPGRADE GUARDIAN, Pluto, and kubent.	Controlled benchmark	Supports false-positive checks for the included safe fixtures.
Pluto and kubent cover the API-deprecation subset in the benchmark.	Pluto/kubent file and cluster runs each matched 4 deprecated-API findings and did not cover the 27 non-API readiness labels.	Controlled benchmark	Supports scoped baseline positioning, not general superiority.
The operator models RBAC gaps as findings.	Local checker test for forbidden API access.	Locally validated	Supported for covered RBAC-denial behavior.
The operator performs bounded-write assessment.	Controller implementation and CRD/status write scope.	Implemented behavior	Supports bounded-write artifact description.

aggregate score was 343 with critical aggregate risk. The generated `UpgradePlan` carried the `DoNotUpgrade` decision, the deterministic recommended order, and 31 non-executing required actions. This result is scenario coverage evidence, not a distribution of risks in real clusters.

C. API-Deprecation Subset

Table VI reports the API-deprecation subset only. KUBEUPGRADE GUARDIAN, Pluto, and kubent all matched the four deprecated-API labels in both file and cluster modes. This table is the fair comparison for Pluto and kubent because their declared purpose is deprecated Kubernetes API detection.

D. Non-API Readiness Coverage

Table VII reports the remaining 27 labeled findings. These labels cover readiness probes, workload availability,

PDB/eviction risks, admission webhook risks, and selected policy-risk indicators. KUBEUPGRADE GUARDIAN covered 27/27 labels with no unexpected findings. Pluto and kubent covered 0/27 because these categories are outside their API-deprecation scope; this is a coverage boundary, not a global superiority claim.

E. KUBEUPGRADE GUARDIAN Metrics by Risk Family

Table VIII reports KUBEUPGRADE GUARDIAN metrics by risk family. The benchmark uses exact resource and message matching for KUBEUPGRADE GUARDIAN labels. Each expected finding specifies type, severity, resource identity, and a message fragment.

F. Negative Controls

Table IX reports the negative-control observations. All tools produced zero observations for the five safe fixtures. For KUBEUPGRADE GUARDIAN, this checks the

TABLE V
LOCAL VALIDATION COVERAGE FROM THE CURRENT GO TEST SUITE.

Area	Covered behavior	Status
Workload availability	Deployment replicas below two are detected with replica evidence	Passed
Readiness probes	Missing container readiness probe is detected	Passed
PDB risk	Single-replica workload with <code>minAvailable: 1</code> is critical	Passed
Admission webhook risk	<code>failurePolicy=Fail</code> and missing webhook service are detected	Passed
Admission selector scope	Nil and empty namespace selectors are treated as broad scope	Passed
Policy risk	Restricted namespace and privileged workload produce findings	Passed
Deprecated API source filter	Legacy last-applied source is detected while modern source manifests served through legacy endpoints are not flagged	Passed
Capacity	Insufficient one-node-loss headroom is detected	Passed
Observability	Missing monitoring namespace is detected	Passed
RBAC gap	Forbidden list call becomes <code>RBAC_ASSESSMENT_GAP</code>	Passed
Controller	Reconciliation creates one idempotent generated plan	Passed
Planner	Required actions and evidence references are generated	Passed
Scoring	Severity weights, risk boundaries, and decision ordering are checked	Passed

TABLE VI
API-DEPRECATION SUBSET METRICS FOR RUN 20260617T091703Z.

Tool	TP	FP	FN	Precision	Recall	F1
KUBEUPGRADE GUARDIAN	4	0	0	1.0000	1.0000	1.0000
Pluto files	4	0	0	1.0000	1.0000	1.0000
Pluto cluster	4	0	0	1.0000	1.0000	1.0000
kubent files	4	0	0	1.0000	1.0000	1.0000
kubent cluster	4	0	0	1.0000	1.0000	1.0000

TABLE VII
NON-API READINESS COVERAGE FOR RUN 20260617T091703Z.

Tool	Covered	Unexpected	Uncovered	Coverage	F1
KUBEUPGRADE GUARDIAN	27	0	0	1.0000	1.0000
Pluto files	0	0	27	0.0000	0.0000
Pluto cluster	0	0	27	0.0000	0.0000
kubent files	0	0	27	0.0000	0.0000
kubent cluster	0	0	27	0.0000	0.0000

TABLE VIII
KUBEUPGRADE GUARDIAN CONTROLLED BENCHMARK METRICS BY RISK FAMILY.

Family	TP	FP	FN	F1
Admission webhook	4	0	0	1.0000
Deprecated API	4	0	0	1.0000
PDB	7	0	0	1.0000
Policy risk	8	0	0	1.0000
Readiness probes	5	0	0	1.0000
Workload availability	3	0	0	1.0000

main false-positive concerns in the benchmark: safe multi-replica workloads should not be treated as availability or readiness risks, fail-open scoped webhooks should not be treated as admission blockers, warn/audit-only Pod Security labels should not be treated as enforced policy blockers, and modern API source manifests should not be flagged solely because the API server can serve legacy versions.

G. Execution Cost

The benchmark records wall-clock durations for the assessment wait loop and baseline commands. KUBE-

UPGRADE GUARDIAN completed the assessment wait in 23.219 seconds after scenario application. Pluto file mode completed in 0.242 seconds and Pluto cluster mode in 1.577 seconds. kubent file mode completed in 0.642 seconds and kubent cluster mode in 6.461 seconds. These numbers are single-run measurements on a local Kind environment; they are reported for reproducibility and relative execution context, not as a scalability claim.

H. Error Analysis

The final controlled run produced 0 KUBEUPGRADE GUARDIAN false positives and 0 KUBEUPGRADE GUARDIAN false negatives against the 31 benchmark labels. The negative-control table produced 0 observations across all five safe fixtures. No per-family residual error remains in the final benchmark output. The main residual risk is benchmark representativeness: the labels cover selected resource patterns and do not exhaust Kubernetes admission, policy, capacity, add-on, or managed-provider behaviors.

TABLE IX
NEGATIVE-CONTROL OBSERVATIONS FOR RUN 20260617T091703Z. PF/PC DENOTE PLUTO FILE/CLUSTER MODE; KF/KC DENOTE KUBENT
FILE/CLUSTER MODE.

Control	Resource	KUG	PF	PC	KF	KC
Safe deployment	profile-api	0	0	0	0	0
Safe webhook	r01-safe-webhook	0	0	0	0	0
Warn/audit-only policy	r01-policy-warn-audit	0	0	0	0	0
Modern HPA	modern-hpa	0	0	0	0	0
Modern PDB	modern-hpa-target	0	0	0	0	0

I. Interpretation Limits

This benchmark is controlled and local. It does not include production workloads, AKS node-pool upgrade settings, CPU/memory profiling, expert actionability ratings, or scoring calibration. The result supports scenario-level detection accuracy, negative-control precision for the included safe fixtures, and scoped baseline comparison. It does not support claims about production effectiveness, statistically robust accuracy across cluster populations, or calibrated risk prediction.

IX. BENCHMARK ARTIFACT AND REPRODUCIBILITY

This section describes the produced replication package for the controlled benchmark. The package separates scenarios, ground truth, tool execution, raw outputs, normalized findings, negative-control observations, and result tables.

A. Produced Artifacts

Table X lists the files used to reproduce the reported results. The runner creates a temporary Kind cluster, installs the CRDs, starts the controller, applies the scenarios, waits for the assessment, runs Pluto and kubent when available on PATH, writes raw outputs, computes normalized metrics, deletes the Kind cluster, and restores the previous kubeconfig context.

B. Replication Command

The controlled benchmark is packaged under `experiments/kind/r01-benchmark`. The run can be reproduced from the repository root with:

```
PATH=/tmp/kug-r01-tools/bin:$PATH \
python3 experiments/kind/r01-benchmark/\
run_kind_benchmark.py \
--operator-repo ../kubeupgrade-guardian-operator \
--restore-context aks-ihsen-mvp-we-admin
```

The runner writes one timestamped directory under `results/`. In the validated run, it restored the kubeconfig context to `aks-ihsen-mvp-we-admin` and deleted the temporary Kind cluster after completion.

X. DISCUSSION

A. Why Evidence Matters

Evidence changes the review conversation. A finding such as “PDB may block disruption” is more useful when it points to the PDB name, selector, matched

workload, replica count, and exact `minAvailable` or `maxUnavailable` value. Similarly, an RBAC-denied list call is not merely a tool error; it is evidence that the assessment cannot establish readiness for a resource family. This makes uncertainty visible.

B. Where the Current Artifact Is Useful

The current artifact is useful as a repeatable pre-upgrade inspection framework, a structured way to expose evidence and assessment gaps, and a basis for comparing broader readiness risk categories against API-deprecation tools. The controlled benchmark shows that the artifact can detect the labeled risks in the current scenario package. Its value is strongest when teams want a Kubernetes-native record of findings and proposed actions without granting the tool authority to mutate workloads.

C. Where the Current Artifact Is Weak

The deprecated-API checker uses a small static table. The observability checker is heuristic. The capacity checker is conservative and does not model all scheduler, autoscaler, topology, taint, priority, or managed-provider behaviors. Policy analysis covers selected indicators and is not a full admission dry run. Add-on and operator compatibility are not part of the measured benchmark. The score is deterministic but not calibrated by production outcomes.

D. Cautious Agent Integration

KAGENT is not part of the evaluated artifact. The measured system is deterministic: checkers produce findings, the scorer produces a risk level, and the planner produces non-executing actions. Any agentic explanation layer would need to remain evidence-bounded and non-authoritative; it is not included in the benchmark results.

XI. THREATS TO VALIDITY

Construct validity. Readiness is multi-dimensional. Precision and recall over labeled findings may not capture business criticality, workload ownership, maintenance-window constraints, or organizational risk tolerance.

Internal validity. Checker defects, incomplete API tables, incorrect benchmark labels, or baseline misconfiguration can bias results. Mitigations include independent scenario review, command archival, deterministic fixtures, and version-pinned tools.

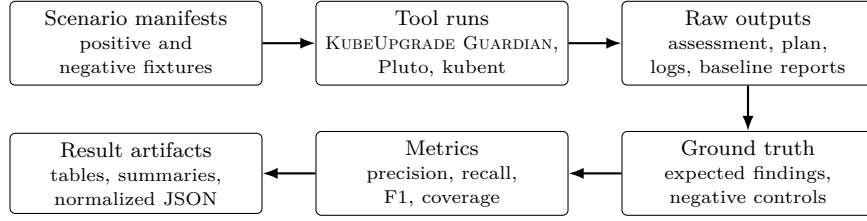


Fig. 3. Controlled benchmark pipeline. The produced package separates scenario inputs, tool execution, raw artifacts, ground truth, metrics, and final result tables.

TABLE X
PRODUCED BENCHMARK ARTIFACTS FOR RUN 20260617T091703Z.

Artifact	Content	Use in results
<code>manifests/00-scenarios.yaml</code>	Kubernetes fixtures for positive and negative scenarios	Benchmark input
<code>ground-truth.json</code>	31 expected findings and 5 negative controls	Label source
<code>run_kind_benchmark.py</code>	Executable benchmark harness	Reproducibility
<code>upgradeassessment.json</code>	Raw <code>UpgradeAssessment</code> status	Finding counts and risk score
<code>upgradeplan.json</code>	Generated <code>UpgradePlan</code> with required actions	Decision and remediation output
<code>normalized-findings.json</code>	Normalized KUBEUPGRADE GUARDIAN, Pluto, and kubent findings	Metric computation
<code>negative-control-observations.json</code>	Per-control observations by tool	False-positive checks
<code>metrics.json</code>	Overall and per-family precision, recall, and F1	Result tables
<code>summary.md</code>	Human-readable run summary	Manuscript extraction

External validity. The controlled Kind benchmark demonstrates behavior on labeled scenarios, not production generality. AKS validation would add managed-cluster evidence but remains provider-specific. Production clusters may include proprietary controllers, custom admission chains, and hidden operational constraints.

Conclusion validity. The current local test suite supports implementation sanity, and the controlled benchmark supports bounded scenario-level accuracy. The data does not support production effectiveness, human-effort reduction, or calibrated risk-prediction claims.

Agent validity. The evaluated artifact does not include KAGENT or another explanation agent. Agentic output is therefore outside all reported metrics.

XII. RELATED WORK

A. Software Evolution and Change Impact Analysis

The framing of upgrades as software evolution builds on Lehman’s view of long-lived software change and later software evolution work [1], [3]. The evidence-guided model is closest to change impact analysis, where the goal is to identify what a change may affect before applying it [2]. KUBEUPGRADE GUARDIAN applies this idea to a live Kubernetes cluster by treating the target Kubernetes version as the change and cluster resources as potentially affected artifacts.

B. DevOps, SRE, and Production Readiness

Continuous delivery and DevOps research provide the operational context: organizations seek faster change while preserving reliability, but adoption introduces technical and organizational challenges [4], [5], [6], [7], [8], [9]. SRE

and production-readiness literature emphasizes reliability, operational feedback, explicit failure-mode handling, and review before risky operations [10], [11]. KUBEUPGRADE GUARDIAN follows this tradition by producing reviewable evidence rather than executing remediations.

C. Infrastructure-as-Code and Kubernetes Configuration Analysis

IaC research shows that infrastructure definitions are software artifacts with defects, security risks, and testing needs [12]. Empirical work on Kubernetes manifests reports configuration and security misconfiguration patterns in real repositories [13], [14]. Recent drift and reconciliation work also frames infrastructure state divergence as a measurable software-engineering problem [46]. These results support the need for assessment tools that reason about concrete configuration and workload state before platform changes.

D. Kubernetes API Deprecation and Upgrade Tooling

Kubernetes official documentation defines the authoritative semantics for API removals, version skew, eviction, PDBs, resource requests, and managed upgrade mechanics [22], [23], [24], [25], [26], [35], [37], [38], [39]. Pluto and kubent specialize in deprecated API detection [40], [41]. Helm documentation and mapkubeapis address Helm-specific API migration issues [42], [43]. KUBEUPGRADE GUARDIAN differs by treating API removals as one risk family among several upgrade-readiness risks.

E. Kubernetes Operators and Operator Reliability

Kubernetes custom resources and the operator pattern define how domain-specific control loops can extend

cluster behavior [32], [33], [48]. Empirical operator-bug studies show that operators can fail through incorrect state observation, reconciliation logic, CRD definitions, and access control [15]. This motivates the manuscript’s implementation-fidelity emphasis: an operator paper must state precisely which resources it reads, which resources it writes, and which controller behaviors are locally tested.

F. Policy-as-Code and Admission Control

Admission controllers, admission webhooks, ValidatingAdmissionPolicy, Pod Security Admission, and RBAC are central to upgrade readiness because recreated or migrated objects may be rejected even when their API versions still exist [28], [27], [29], [34], [30], [31]. The Kubernetes admission-control threat model further emphasizes that admission components are security-sensitive operational dependencies [36]. Policy-as-code systems such as OPA make policy enforcement programmable, and recent empirical work studies how policy-as-code is adopted in open-source projects [44], [45]. KUBEUPGRADE GUARDIAN currently performs scoped policy and admission checks; it does not implement a full admission dry run or a complete policy-engine semantics model.

G. AIOps and LLM-Based Operations Assistance

AIOps research studies operational automation, incident management, and failure management at scale [16], [17]. LLM-for-software-engineering surveys and empirical security studies show both opportunities and risks: AI assistance can help with explanation and synthesis, but it may also produce unsupported or insecure output and encourage overconfidence [18], [19], [20], [21]. This motivates the paper’s cautious stance on KAGENT: an agent layer should be optional, evidence-bounded, non-authoritative, and separately evaluated.

XIII. CONCLUSION

This paper reconstructs Kubernetes upgrade readiness as evidence-guided impact analysis over live cluster state. KUBEUPGRADE GUARDIAN implements this idea as a Kubernetes-native bounded-write operator that materializes assessments and plans through `UpgradeAssessment` and `UpgradePlan`. The current artifact includes deterministic checkers, structured evidence, scoring, decisions, and non-executing remediation actions. Local validation completed with `go test ./...` and 0 failures. The controlled Kind benchmark run `20260617T091703Z` reported 31 true positives, 0 false positives, and 0 false negatives for KUBEUPGRADE GUARDIAN against the labeled scenario package, with 0 observations on 5 negative controls. Pluto and kubent matched all 4 deprecated-API findings in the API subset, while the 27 non-API readiness labels remained outside their scope. The reported evidence supports the bounded claim that KUBEUPGRADE GUARDIAN implements and detects the evaluated readiness patterns in the controlled benchmark.

REFERENCES

- [1] M. M. Lehman, “Programs, life cycles, and laws of software evolution,” *Proceedings of the IEEE*, vol. 68, no. 9, pp. 1060–1076, 1980, doi: 10.1109/PROC.1980.11805.
- [2] S. A. Bohner and R. S. Arnold, Eds., *Software Change Impact Analysis*. Los Alamitos, CA, USA: IEEE Computer Society Press, 1996.
- [3] T. Mens and S. Demeyer, Eds., *Software Evolution*. Berlin, Heidelberg: Springer, 2008, doi: 10.1007/978-3-540-76440-3.
- [4] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.
- [5] L. Chen, “Continuous delivery: Huge benefits, but challenges too,” *IEEE Software*, vol. 32, no. 2, pp. 50–54, 2015, doi: 10.1109/MS.2015.27.
- [6] M. Shahin, M. A. Babar, and L. Zhu, “Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices,” *IEEE Access*, vol. 5, pp. 3909–3943, 2017, doi: 10.1109/ACCESS.2017.2685629.
- [7] J. Smeds, K. Nybom, and I. Porres, “DevOps: A definition and perceived adoption impediments,” in *Agile Processes in Software Engineering and Extreme Programming*, LNBP 212. Springer, 2015, pp. 166–177, doi: 10.1007/978-3-319-18612-2_14.
- [8] L. A. F. Leite, C. Rocha, F. Kon, D. Milojicic, and P. Meirelles, “A survey of DevOps concepts and challenges,” *ACM Computing Surveys*, vol. 52, no. 6, 2019, doi: 10.1145/3359981.
- [9] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.
- [10] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., *Site Reliability Engineering: How Google Runs Production Systems*. O’Reilly Media, 2016.
- [11] M. T. Nygard, *Release It!: Design and Deploy Production-Ready Software*, 2nd ed. Pragmatic Bookshelf, 2018.
- [12] A. Rahman, R. Mahdavi-Hezaveh, and L. Williams, “A systematic mapping study of infrastructure as code research,” *Information and Software Technology*, vol. 108, pp. 65–77, 2019, doi: 10.1016/j.infsof.2018.12.004.
- [13] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, “Security misconfigurations in open source Kubernetes manifests: An empirical study,” *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 4, article 99, 2023, doi: 10.1145/3579639.
- [14] Y. Zhang, U. Paul, M. d’Amorim, and A. Rahman, “Configuration defects in Kubernetes,” arXiv:2512.05062, 2025. [Online]. Available: <https://arxiv.org/abs/2512.05062>
- [15] Q. Xu, Y. Gao, and J. Wei, “An empirical study on Kubernetes operator bugs,” in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1746–1758, doi: 10.1145/3650212.3680396.
- [16] Y. Dang, Q. Lin, and P. Huang, “AIOps: Real-world challenges and research innovations,” in *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings*, 2019, pp. 4–5, doi: 10.1109/ICSE-Companion.2019.00023.
- [17] P. Notaro, J. Cardoso, and M. Gerndt, “A survey of AIOps methods for failure management,” *ACM Transactions on Intelligent Systems and Technology*, vol. 12, no. 6, 2021, doi: 10.1145/3483424.
- [18] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, “Large language models for software engineering: A systematic literature review,” *ACM Transactions on Software Engineering and Methodology*, 2024, doi: 10.1145/3695988.
- [19] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, “Do users write more insecure code with AI assistants?” arXiv:2211.03622, 2022. [Online]. Available: <https://arxiv.org/abs/2211.03622>
- [20] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, “Asleep at the keyboard? Assessing the security of GitHub Copilot’s code contributions,” arXiv:2108.09293, 2021. [Online]. Available: <https://arxiv.org/abs/2108.09293>
- [21] Y. Fu, P. Liang, A. Tahir, Z. Li, M. Shahin, J. Yu, and J. Chen, “Security weaknesses of Copilot-generated code in GitHub projects: An empirical study,” 2025, doi: 10.1145/3716848.

- [22] Kubernetes Documentation, “Kubernetes deprecation policy.” [Online]. Available: <https://kubernetes.io/docs/reference/using-api/deprecation-policy/>
- [23] Kubernetes Documentation, “Deprecated API migration guide.” [Online]. Available: <https://kubernetes.io/docs/reference/using-api/deprecation-guide/>
- [24] Kubernetes Documentation, “Version skew policy.” [Online]. Available: <https://kubernetes.io/releases/version-skew-policy/>
- [25] Kubernetes Documentation, “API-initiated eviction.” [Online]. Available: <https://kubernetes.io/docs/concepts/scheduling-eviction/api-eviction/>
- [26] Kubernetes Documentation, “Disruptions.” [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/pods/disruptions/>
- [27] Kubernetes Documentation, “Dynamic admission control.” [Online]. Available: <https://kubernetes.io/docs/reference/access-authn-authz/extensible-admission-controllers/>
- [28] Kubernetes Documentation, “Admission controllers reference.” [Online]. Available: <https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/>
- [29] Kubernetes Documentation, “Validating admission policy.” [Online]. Available: <https://kubernetes.io/docs/reference/access-authn-authz/validating-admission-policy/>
- [30] Kubernetes Documentation, “Using RBAC authorization.” [Online]. Available: <https://kubernetes.io/docs/reference/access-authn-authz/rbac/>
- [31] Kubernetes Documentation, “Role based access control good practices.” [Online]. Available: <https://kubernetes.io/docs/concepts/security/rbac-good-practices/>
- [32] Kubernetes Documentation, “Custom resources.” [Online]. Available: <https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/>
- [33] Kubernetes Documentation, “Operator pattern.” [Online]. Available: <https://kubernetes.io/docs/concepts/extend-kubernetes/operator/>
- [34] Kubernetes Documentation, “Pod security admission.” [Online]. Available: <https://kubernetes.io/docs/concepts/security/pod-security-admission/>
- [35] Kubernetes Documentation, “Resource management for pods and containers.” [Online]. Available: <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>
- [36] Kubernetes SIG Security, “Kubernetes admission control threat model.” [Online]. Available: <https://github.com/kubernetes/sig-security/blob/main/sig-security-docs/papers/admission-control/kubernetes-admission-control-threat-model.md>
- [37] Microsoft Learn, “Upgrade options and recommendations for Azure Kubernetes Service (AKS) clusters.” [Online]. Available: <https://learn.microsoft.com/en-us/azure/aks/upgrade-options>
- [38] Microsoft Learn, “Upgrade Azure Kubernetes Service (AKS) node images.” [Online]. Available: <https://learn.microsoft.com/en-us/azure/aks/upgrade-node-image>
- [39] Microsoft Learn, “AKS day-2 guide: Patch and upgrade guidance.” [Online]. Available: <https://learn.microsoft.com/en-us/azure/architecture/operator-guides/aks/aks-upgrade-practices>
- [40] FairwindsOps, “Pluto: A CLI tool to help discover deprecated Kubernetes apiVersions,” GitHub repository. [Online]. Available: <https://github.com/FairwindsOps/pluto>
- [41] DoiT International, “kube-no-trouble (kubent),” GitHub repository. [Online]. Available: <https://github.com/doitintl/kube-no-trouble>
- [42] Helm Documentation, “Deprecated Kubernetes APIs.” [Online]. Available: https://helm.sh/docs/topics/kubernetes_apis/
- [43] Helm, “helm-mapkubeapis,” GitHub repository. [Online]. Available: <https://github.com/helm/helm-mapkubeapis>
- [44] Open Policy Agent, “Open Policy Agent documentation.” [Online]. Available: <https://www.openpolicyagent.org/docs/>
- [45] P. L. Foale, F. Khomh, L. Da Silva, and E. Merlo, “An empirical study of policy-as-code adoption in open-source software projects,” arXiv:2601.05555, 2026. [Online]. Available: <https://arxiv.org/abs/2601.05555>
- [46] Z. Yang, H. Guan, V. Nicolet, B. Paulsen, J. Dodds, D. Kroening, and A. Chen, “Automated cloud infrastructure-as-code reconciliation with AI agents,” arXiv:2510.20211, 2025. [Online]. Available: <https://arxiv.org/abs/2510.20211>
- [47] kagent-dev, “kagent: Cloud native agentic AI,” GitHub repository. [Online]. Available: <https://github.com/kagent-dev/kagent>
- [48] CNCF TAG App Delivery Operator Working Group, “Operator white paper.” [Online]. Available: https://github.com/cncf/tag-app-delivery/blob/main/operator-wg/whitepaper/Operator-WhitePaper_v1-0.md